

✓ # Practical 2

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd
import numpy as np
```

✓ step 1: Load the data from CSV file and split it into training and test datasets

```
data = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/
```

```
x = data.drop(columns = ['Outcome'])
y = data['Outcome']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
data.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPec
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

Next steps:

[Generate code with data](#)

[New interactive sheet](#)

✓ Step 2: Summarize the properties in training datasets so that we can calculate probabilities and make predictions.

```
data.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100

```
summary_stats_per_class = x_train.copy()
summary_stats_per_class['Outcome'] = y_train
summary_stats = summary_stats_per_class.groupby('Outcome').agg(['mean', 'std'])
summary_stats
```

Next steps:

[Generate code with summary_stats](#)[New interactive sheet](#)

Step 3: Classify samples from a test dataset and a summarized training dataset

```
from inspect import classify_class_attrs
nb_classifier = GaussianNB()
nb_classifier.fit(x_train, y_train)
y_pred = nb_classifier.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
accuracy, classification_rep
```

```

(0.7662337662337663,
',
precision recall f1-score support\n\n
0.83 0.80 0.81 99\n 1 0.66 0.71 0.68
55\n\n accuracy 0.77 154\n macro avg
0.75 0.75 0.75 154\nweighted avg 0.77 0.77 0.77
154\n')
```